

Compiler Design

A clean study sheet from the handwritten scans

PAGE-02

SYLLABUS

Syllabus of Compiler Design (R22) - Unit I

1. Introduction

- Structure of a compiler
- Science of building a compiler
- Programming language basics

2. Lexical Analysis

- Role of the lexical analyzer
- Input buffering and recognition of tokens
- Lex, finite automata, and regular expressions
- Regular expressions to automata
- Design and optimization of DFA-based pattern matchers

Introduction

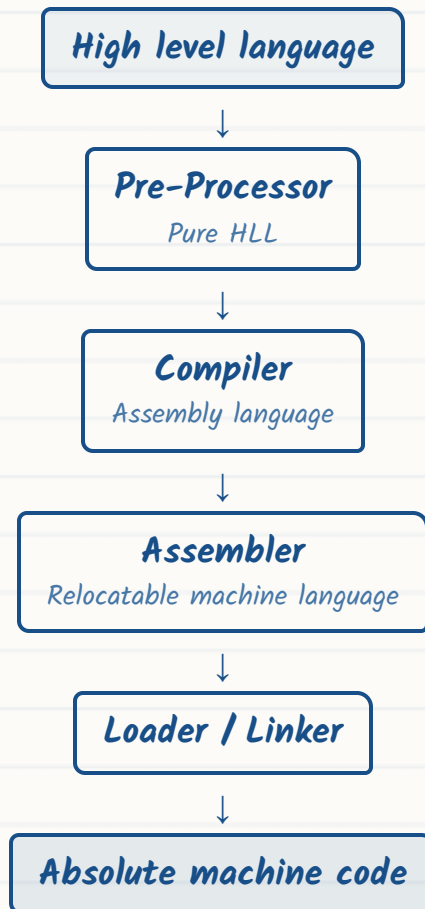
A compiler converts a high-level language written by a programmer into a low-level object, target, or machine language that can be understood by a computer.



1. Language Processing Systems

The language processing system is a collection of programs that work together to translate source code into executable machine code. Each stage transforms the program into a form that the next stage can understand.

Overview of Language Processing System



1. High Level Language

- Written using human-readable statements and programming constructs.
- #define and #include are processed before compilation.
- These instructions are called preprocessor directives.

Preprocessor / Assembly Language / Assembler

1. Preprocessor

- Processes preprocessor directives such as `#define` and `#include`.
- Produces a pure high-level language program for the compiler.

2. Assembly Language

- A symbolic, low-level language generated by the compiler.
- Uses mnemonics to represent machine operations.

3. Assembler

- Translates assembly language into relocatable machine language.

Relocatable Machine Code

Machine code whose addresses can be adjusted when the program is loaded into memory.

Loader / Linker

- Converts relocatable code into absolute code and tries to run the program, resulting in a running program or an error message (sometimes both can happen).
- The linker loads a variety of object files into a single file to make it executable.
- The loader then loads it into memory and executes it.

1. Compilation: Two Parts

Compilation can be split into two parts: analysis and synthesis.

2. Analysis (Front End)

- The source program is read and broken down into pieces.
- Determines the syntax and meaning of the source string.
- An intermediate code is generated.
- Carried out by lexical analysis, syntax analysis, and semantic analysis.

Synthesis (Back End)

→The intermediate code is optimized if needed.

→It is then converted into an equivalent target program.



1. Properties of a Good Compiler

→Must be bug-free.

→Must generate correct machine code.

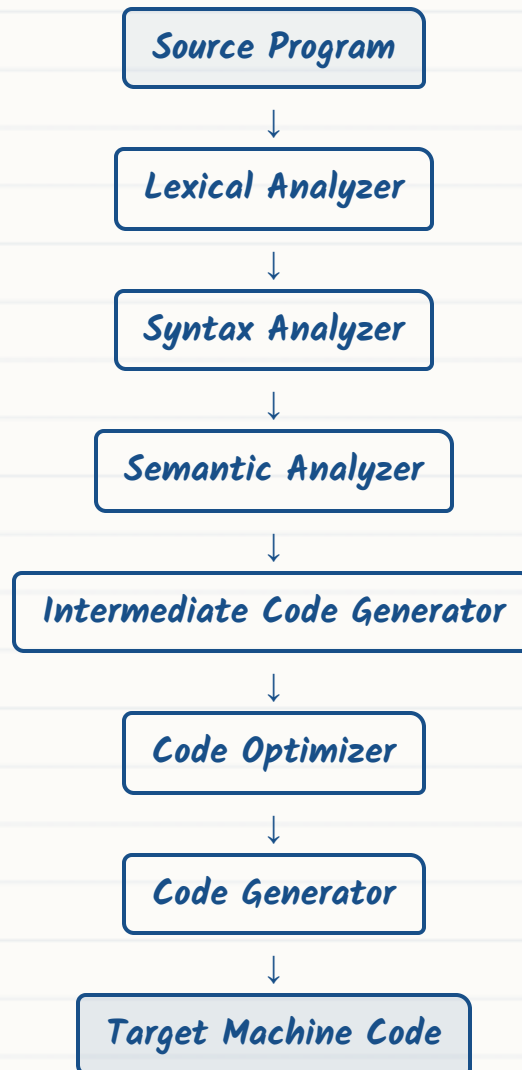
→Must be portable.

→Must give good diagnosis and error messages.

→Generated code must work well with existing debuggers.

→Must have consistent optimization.

Phases of a Compiler



Symbol Table & Error Handling

Symbol table management records identifiers and their attributes, and error detection & handling is invoked whenever a mistake is found. Both run alongside every phase above.

1. Analysis Phase

→Lexical analyzer divides the program into tokens.

- Syntax analyzer recognizes "sentences" in the program using the syntax of the language.
- Semantic analyzer checks the semantics of each construct.

2. Synthesis Phase

- Intermediate code is easy to generate and easy to translate further.
- Code optimizer improves the intermediate code to increase efficiency.
- Code generator produces the target code.

Lexical Analyzer

→ Lexical analysis is also called scanning.

→ It collects sequences of characters into meaningful units called tokens, like words of a natural language, e.g. "I am hungry".

→ A token is a sequence of characters having a collective meaning.

As an example, consider the line of code `a[index] = 4 + 2`. It contains 8 tokens:

TOKEN	KIND
a	identifier
[	left bracket
index	identifier
]	right bracket
=	assignment
4	number
+	plus sign
2	number

Whitespace

Blank characters used in programming statements are eliminated during the lexical analysis phase.

Each token consists of one or more characters collected into a unit before further processing takes place.

Syntax Analyzer

→ Also called parsing.

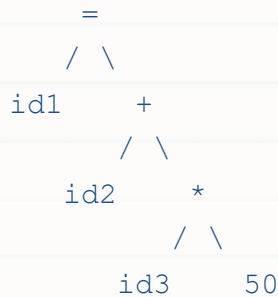
→ The parser receives the source code as tokens from the scanner and performs syntax analysis, which determines the structure of the program.

→ Similar to checking grammar in a natural-language sentence.

→ Determines the structural elements of the program as well as their relationships.

→ Results are usually represented as a parse tree or syntax tree, in hierarchical form.

For example, the token stream $id1 = id2 + id3 * 50$ is represented by the following syntax tree:



Semantic Analyzer

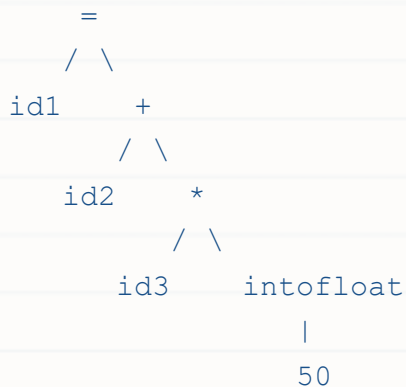
- The third phase of the compilation process.
- Verifies whether the parse tree is meaningful — checks that it follows the rules of the language.
- Keeps track of identifiers, their types, and expressions.

1. Type Checking

Checks the operands for all operators, e.g. $a + b$ where the operands are a float and an int.

2. Coercion

Permits certain type conversions. Example: $Sum := oldsum + Rate * 50$ becomes $id1 := id2 + id3 * 50$. Since $id1$, $id2$, and $id3$ are floating-point numbers but 50 is an integer, coercion takes place to convert 50 to a float.



Intermediate Code Generator

- Generates intermediate code in a form that can be readily executed by a machine.
- Easy to generate and easy to convert into target code.
- Generated between the high-level language and the machine language.
- The compiler must generate a temporary name to hold the value computed by each instruction.

The syntax tree generated during semantic analysis is converted into intermediate code:

```
t1 = intofloat(50)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Code Optimizer

- Improves the intermediate code for faster execution and lower memory consumption.
- Removes unnecessary lines of code and reorders statements to speed up program execution.

Intermediate code:

```
t1 = intofloat(50)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Optimized code:

```
t1 = id3 * 50.0
id1 = id2 + t1
```

Code Generator

- Main purpose is to write code that the target machine can understand.
- The final stage of the compilation process.
- Takes the optimized intermediate code as input and maps it to the target machine language.
- Translates the intermediate code into a sequence of machine instructions.

Optimized code:

```
t1 = id3 * 50.0  
id1 = id2 + t1
```

Code generation:

```
MOVF id3, R2  
MULF #50.0, R2  
MOVF id2, R2  
ADDF R2, R1  
MOVF R1, id1
```

Compiler vs Interpreter

1. Compiler

- A translator that takes a high-level language as input and produces a low-level (machine or assembly) language as output.
- More intelligent than an assembler — checks limits, ranges, errors, etc.
- Slower: it goes through the entire program before translating it into machine code, so run time is longer and it occupies more memory.



2. Interpreter

- Translates a programming language into a comprehensible language, one statement at a time.
- Interpreters are, more often than not, smaller than compilers.



Difference between Compiler and Interpreter

COMPILER	INTERPRETER
Scans the whole program in one go.	<i>Translates the program one statement at a time.</i>
Errors are shown at the end, together (if any).	<i>Errors are shown line by line.</i>
Localization of errors is difficult.	<i>Localization of errors is easy.</i>
Consumes less time — faster than an interpreter.	<i>Consumes more time — slower than a compiler.</i>
More efficient.	<i>Less efficient.</i>
CPU utilization is more.	<i>CPU utilization is less.</i>
Both syntactic and semantic errors can be checked simultaneously.	<i>Only syntactic errors are checked.</i>
The compiler is larger.	<i>Interpreters are often smaller than compilers.</i>
Not flexible.	<i>Flexible.</i>
Examples: C, C++, etc.	<i>Examples: Python, Ruby, Perl, SNOBOL, etc.</i>

The Science of Building a Compiler

- Building a compiler is a challenging task. Its main job is to accept a source program of any size and convert it into a suitable target program.
- Compiler study focuses on how to design the correct mathematical model and choose correct algorithms, balancing the need for generality and efficiency.
- Finite state machines and regular expressions (already studied in FLAT) are the prerequisites for describing the lexical units of programs (tokens) and the algorithms used by the compiler to identify these tokens.
- The term "code optimization" indicates the attempts made by a compiler to produce code that is more efficient than the previous code — faster than any other code that performs the same task.

Objectives of Compiler Optimization

- The meaning of the compiled program must be preserved.
- Optimization should improve program performance.
- Time required for compilation should be reasonable.
- The engineering effort required must be manageable.

1. Programming Language Basics

The topics that make up programming language basics:

- Static/dynamic distinction
- Environments and states
- Static scope and block structure
- Explicit access control
- Dynamic scope
- Declarations and definitions
- Parameter passing mechanisms
- Aliasing

Static / Dynamic Distinction

Among the most important issues when designing a compiler for a language is "what decisions can the compiler make about a program". If a language uses a policy that allows the compiler to decide an issue, the language uses a **static policy** — the issue can be decided at compile time.

```
static void main()
{
    int num1, num2;
    int sum = num1 + num2    <- no semicolon: error
}                            occurs at compile time
```

If the decision is made at run time, the language uses a **dynamic policy**.

```
public void main()
{
    int a = 0;
    int num = 10/a;    <- error at run time
}
```

Environment & States

The environment is a mapping from names to locations in the store. Since variables refer to locations, we could alternatively define an environment as a mapping from names to variables.

The state is a mapping from locations in the store to their values.



Static Scope and Block Structure

- The scope rules for C are based on program structure.
- The scope of a variable is implicitly determined by where the declaration appears in the program.
- Languages like C++, C#, and Java provide explicit control over scope through keywords like public, private, and protected.
- Static scope can be given by using blocks. C uses braces { } to denote a block; Algol uses begin and end.
- Blocks can be nested inside other blocks.
- A variable declaration D belongs to a block B if it is declared in B. If the declaration D of name x is in B, then the scope of x is B as well as any nested block.

Nested Blocks Example

Note

x will not have scope in a nested block if it is redeclared in that block.

Consider the program below, which has 4 blocks with several definitions of a and b. Each declaration initializes its variable to the number of the block to which it belongs.

```
main
{
    // B1
    int a = 1;
    int b = 1;
    {
        // B2
        int b = 2;
        {
            // B3
            int a = 3;
            cout << a << b;
        }
        {
            // B4
            int b = 4;
            cout << a << b;
        }
        cout << a << b;
    }
    cout << a << b;
}
```

Declarations and Their Scopes

DECLARATION	SCOPE
<code>int a = 1</code>	<i>B1 - B3</i>
<code>int b = 1</code>	<i>B1 - B2</i>
<code>int b = 2</code>	<i>B2 - B4</i>
<code>int a = 3</code>	<i>B3</i>
<code>int b = 4</code>	<i>B4</i>

Consider the print statement of B4: since B4 has redeclared `b = 4`, it prints the `b` value as 4. It does not contain `a`; its surrounding block B2 also does not contain an `a` declaration. B2's surrounding block B1 has an `a` declaration, so the `a` value is printed as 1.

Explicit Access Control

Through the use of keywords like public, private, and protected, object-oriented languages such as C++ or Java provide explicit control over access to member names in a superclass. These keywords support encapsulation by restricting access.

- Private names have a scope that includes only the method declarations and definitions associated with that class or any friend class.
- Protected names are accessible to subclasses.
- Public names are accessible by anyone from outside.

Dynamic Scope / Declarations & Definitions

1. Dynamic Scope

- Dynamic scoping is a programming language paradigm that you don't typically see.
- Any scoping policy is dynamic if it is based on factors that can be known only when the program executes (at running time).

2. Declarations and Definitions

Declarations tell us about the type of things, while definitions tell us about their values.

```
example:  int i = 1;
```

```
int i      => Declaration
```

```
i = 1      => Definition
```


Parameter Passing Mechanisms

We will see how the actual parameters (the real values that will be called for the procedure) are associated with the formal parameters (which define the procedure, i.e. like a formula).

There are different mechanisms for calling the parameters. The majority of languages use either "call by value" or "call by reference", or both. There are 3 types of parameter passing mechanisms:

→ Call-by-Value

→ Call-by-Reference

→ Call-by-Name

Call-by-Value

The value of the actual parameter, in "read only mode", is transmitted to the formal parameters.

```
#include <iostream>
using namespace std;

void show(int x)          // formal parameter
{
    cout << x << endl;
}

int main()
{
    int age = 20;
    show(age);             // default parameter
    show(10);              // actual parameter
    return 0;
}
```

O/P: 20
10

Call-by-Reference

The reference / address of the actual parameter is transmitted to the formal parameters.

```
#include <iostream>
using namespace std;

void show(int *x)
{
    cout << *x << endl;
}

int main()
{
    int age = 20;
    show(&age);
    return 0;
}
```

O/P: 20

Call-by-Name

This technique is used in programming languages such as Algol. The symbolic "name" of a variable is passed, which allows it both to be accessed and updated.

Example: to double the value of c[j], you can pass its name (not its value) into the following procedure:

```
procedure double(x);  
  real x;  
  begin  
    x := x * 2  
  end;
```

Aliasing

An alias occurs when different variables point directly or indirectly to a single area of storage.

```
A ---\  
B ----+---> [ same storage ]  
C ---/
```

Lexical Analysis

1. The Role of the Lexical Analyzer

→ Lexical analysis is the first phase of the compiler.

→ It reads input characters and produces output as a sequence of tokens that the parser uses for syntax analysis.



2. Input Buffering

Lexical analysis has to access secondary memory each time to identify tokens, which is time-consuming and costly. So the input strings are stored into a buffer and then scanned by the lexical analyzer.

Scanning with Two Pointers

Lexical analysis scans the input string from left to right, one character at a time, to identify tokens. It uses two pointers to scan tokens:

→ **Begin pointer (bptr)** — points to the beginning of the string to be read.

→ **Look-ahead pointer (lptr)** — moves ahead to search for the end of the token.

Example: for the statement int a, b;

```
bptr
v
[ i | n | t |   | a | , | b | ; ]
^
lptr
```

The look-ahead pointer scans the buffer until the token is found.

Recognizing a Token

The character ("blank space") beyond the token `int` has to be examined before the token `int` is determined.

```

bptr
v
[ i | n | t |   | a | , | b | ; ]
          ^
          lptr

```

After processing the token `int`, both pointers are set to the next token (`a`), and this process is repeated for the whole program.

```

                bptr
                v
[ i | n | t |   | a | , | b | ; ]
'--token--'    ^
                lptr

```

Buffer Halves

A buffer can be divided into two halves.

→ *If the look-ahead pointer moves toward halfway in the first half, the second half is filled with new characters.*

→ *If the look-ahead pointer moves toward the right end of the buffer (the second half), the first half is filled with new characters, and it goes on...*

```
bptr
v
[ ..... | ..... ]
          ^
          lptr
'--first--' '--second--'
    half      half
```


Buffering Techniques

1. Buffer Pairs

A specialized buffering technique can decrease the amount of overhead needed to process an input character when transferring characters. It includes two buffers, each of N-character size, which are reloaded alternatively.

2. Sentinels

In the buffer-pair technique, each time we have to examine for the end of a half of the buffer. To avoid that time waste, in the sentinels technique each half of the buffer holds a sentinel character at the end to identify the buffer ending. Generally eof is used as the sentinel character.

[... | ... | eof | ... | ... | eof]

Token, Lexeme, Pattern

1. Token

A token is a sequence of characters that can be treated as a single logical entity. Typical tokens are:

→ Identifiers

→ Keywords

→ Operators

→ Special symbols

→ Constants

2. Pattern

A pattern denotes the type / form of the lexemes of a token. For example, if a token is a keyword, the pattern is the characters that form the keyword.

Lexeme

A lexeme is a sequence of characters in the source program that matches the pattern for a token.

1. Example 1

	VALUE
Token	<i>comparison</i>
Lexeme	<i><=, !=, >=, ==</i>
Pattern	<i><, >, =, !</i>

2. Example 2

	VALUE
Token	<i>keyword</i>
Lexeme	<i>void</i>
Pattern	<i>v, o, i, d</i>

Recognition of Tokens

The input string is compared with patterns and tokens are generated. For example, consider the grammar below:

```
stmt -> if expr then stmt
      | if expr then stmt else stmt
      | ε

expr -> term relop term
      | term

term -> id
```

The above grammar is a simple fragment of branching and conditional statements. This syntax is similar to Pascal. relop is a comparison operator like = for equal and <> for not equal in Pascal.

Patterns via Regular Definitions

The terminals of the grammar — *if*, *then*, *else*, *relop*, *id*, and *number* — are the names of the tokens. The patterns for the tokens are described using regular definitions:

```
digit  -> [0-9]
digits -> digit+
number -> digit.(digit)?(e.[+-]?digit)?
letter -> [A-Z, a-z]
id      -> letter(letter|digit)*
if      -> if
then    -> then
else    -> else
relop   -> < | > | <= | >= | == | <>
```

The Lexical Analyzer Generator — Lex

- Lex is a program that generates lexical analyzers. It is used with the YACC parser generator.
- The lexical analyzer is a program that transforms an input stream into a sequence of tokens.
- It reads the input stream and produces the source code as output through implementing the lexical analyzer in a C program.

1. Function of Lex — Step 1

Firstly, a program lex.l is created in the Lex language. Then the Lex compiler runs the lex.l program and produces a C program lex.yy.c.

Lex — Steps 2 and 3



Step 2: Finally, the C compiler runs the lex.yy.c program and produces an object program a.out.



Step 3: a.out is the lexical analyzer that transforms an input stream into a sequence of tokens.



Program Structure of Lex

In the input file, there are 3 sections:

→ *Definition section*

→ *Rules section*

→ *User code section*

1. Definition Section

The definition section contains the declaration of variables, regular definitions, and manifest constants. Text is enclosed in `%{ ... %}` brackets. Anything written in these brackets is copied directly to the file `lex.yy.c`.

```
%{  
    // Definitions  
%}
```


Rules Section

The rules section contains a series of rules in the form pattern action: the pattern must be unindented, and the action begins on the same line inside { } brackets. The rules section is enclosed in %% ... %%. -----

```
%%
pattern action
%%
```

Examples — the table below shows some of the pattern matches:

PATTERN	IT CAN MATCH WITH
[0-9]	all the digits between 0 and 9
[^a]	all the other characters except a
[^A-Z]	all the other characters except the uppercase letters
[a-z]	all lowercase letters

User Code Section

This section contains C statements and additional functions. We can also compile these functions separately and load them with the lexical analyzer.

1. Overall Basic Program Structure

```
%{  
    // Definitions      <- Definition section  
%}  
  
%%  
Rules                  <- Rules section  
%%  
  
User code section
```

Finite Automata — Transition Diagrams

An intermediate step in the lexical analyzer: we first convert patterns into flowcharts, called "transition diagrams".

- A transition diagram has a collection of nodes or circles, called states.*
- Each state represents a condition that could occur during the process of scanning the input for a lexeme matching a pattern.*
- A state is a summary between the lexemeBegin pointer and the forward pointer.*
- Edges direct from one state of the transition diagram to another. Every edge is labelled by one or more symbols.*
- Example: if we are in state s and the next input symbol is a , we look for an edge out of s labelled by a . If such an edge is found, we advance.*

Transition Diagram Conventions

We advance the forward pointer and enter the state where the edge leads. Some important conventions about transition diagrams:

1. Accepting / Final States

- These states indicate that a lexeme has been found.
- The accepting / final state is represented by a double circle.
- If any action is to be taken after this state — like returning a token and attribute value to the parser — it will be attached to the accepting state.

2. Retraction

If it is necessary to retract (move the forward pointer backwards) one position, we place a star (*) near the accepting state.

Transition Diagram for relop

One state is designated as the "start state" or "initial state", indicated by the label start entering from nowhere. The figure below shows a transition diagram that recognizes lexemes matching the token relop (relational operator).

→ We begin at state 0.

→ If the input symbol is <, then among the lexemes matching the pattern for relop we could see <, <>, or <= — we therefore go to state 1 and look at the next character.

→ If the next character is =, we recognize <=, enter state 2, and return the token relop with attribute LE.

→ If in state 1 the next character is >, then we have the lexeme <>, enter state 3, and return the not-equals operator token.

relop Diagram (continued)

On any other character, the lexeme is `<`, and we enter state 4 and return that information. State 4 has `*` to indicate that we must retract the input one position.

```

start    <      =
--> (0) ---> (1) ---> ((2))  return(relop, LE)
      |      | >
      |      +----> ((3))  return(relop, NE)
      |      | other
      |      +----> ((4))*  return(relop, LT)
      | =
      +-----> ((5))  return(relop, EQ)
      | >      =
      +----> (6) ---> ((7))  return(relop, GE)
              | other
              +----> ((8))*  return(relop, GT)

```

Finite Automata

Lex is a tool which converts an input program to a lexical analyzer. The heart of the transition is known as finite automata.

→ *Finite automata are graphs, like transition diagrams.*

→ *Finite automata are recognizers: they say "yes" or "no" about each possible string.*

→ *Finite automata come in two kinds: DFA (deterministic finite automata) and NFA (non-deterministic finite automata).*

1. Deterministic Finite Automata

A DFA has a 5-tuple notation:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q = set of states used in the FA

Σ = input symbols

δ = transition function (moving from one state to another)

q_0 = initial state

DFA — Example

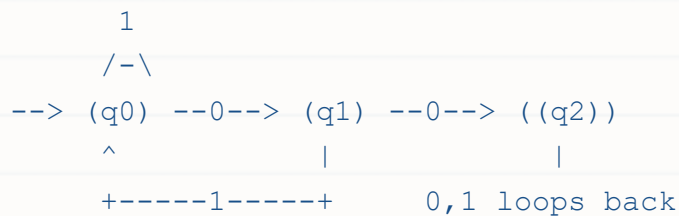
F = final state

$$\delta : Q \times \Sigma \rightarrow Q$$

Note

From the given current state, by taking an input the DFA moves to exactly one state (a unique path), and it does not accept null values.

Example: design a DFA that ends with input symbols $\{1, 0\}$:



Checking the string 11100 with the transition function:

```

δ(q0, 11100) |- δ(q0, 1100)
                |- δ(q0, 100)
                |- δ(q0, 00)
                |- δ(q1, 0)
                |- q2    ✓ accepted

```


DFA Transition Table

STATE	0	1
q0	q1	q2
q1	q2	q0
q2	q0	q0

1. Language Example

Consider the language $L = \{(a|b)^* abb\}$ with $\Sigma = \{a, b\}$. Here $a|b$ means a or b — we can take any number of a 's and b 's, and $*$ is the Kleene closure (includes the null string).

$L = \{ abb, aabb, babb, \dots \}$

abb: --> (q0) -a-> (q1) -b-> (q2) -b-> ((q3))

NFA — Non-Deterministic Finite Automata

An NFA is represented essentially like a DFA. The NFA also has a five-tuple notation:

$$A = (Q, \Sigma, \delta, q_0, F)$$

Q = finite set of states

Σ = input symbols

q_0 = initial state

F = final state

δ = transition function: $\delta : Q \times \Sigma \rightarrow 2^Q$

Notice that the only difference between an NFA and a DFA is in the type of value that δ returns: a set of states in the case of an NFA, and a single state in the case of a DFA.

NFA — Example

Consider the language $L = \{a^i b^j \mid i \geq 1, j \geq 1\}$, $\Sigma = \{a, b\}$:

```

a,b
/-\
--> (q0) -a-> (q1) -b-> (q2) -b-> ((q3))

```

NFA Transition Table

STATE	A	B
q0	q0, q1	q0
q1	—	q2
q2	—	q3
q3	—	—

Regular Expressions to Automata

Regular expressions are a choice of notation to describe lexical analyzers and other pattern-processing software.

Implementation of such software requires the simulation of a DFA. Because an NFA has a choice of input symbol or an ϵ , an NFA's simulation is less straightforward than a DFA's. Thus it is often important to convert an NFA to a DFA that accepts the same language.

Algorithm

Subset construction of a DFA from an NFA (the subset construction method).

Subset Construction — NFA for $(a/b)^*abb$

The figure shows an NFA accepting $(a/b)^*abb$; we convert it into a DFA. States 0-10 are connected with ϵ -transitions and the a/b edges of the pattern.

ϵ -closure(0) is $\{0, 1, 2, 4, 7\}$ — these are the states reachable from 0 on ϵ -transitions. This set is equivalent to state A in the DFA. The input alphabet is $\Sigma = \{a, b\}$. We need to find ϵ -closure(move(A, a)) — i.e. $Dtrans[A, a]$ — and ϵ -closure(move(A, b)) — i.e. $Dtrans[A, b]$.

```

 $\delta(A, a) = \epsilon\text{-closure}(\delta(\{0, 1, 2, 4, 7\}, a))$ 
           =  $\epsilon\text{-closure}(\delta(0, a), \delta(1, a), \delta(2, a), \delta(4, a), \delta(7, a))$ 
           =  $\epsilon\text{-closure}(3, 8) = \{1, 2, 3, 4, 6, 7, 8\}$ 

```

```

 $Dtrans[A, a] = \epsilon\text{-closure}(3, 8) = \{1, 2, 3, 4, 6, 7, 8\}$ 
    let's call this state B.

```

Subset Construction — States B and C

So $Dtrans[A, a] = B$. Now for $Dtrans[A, b]$: among the states of A, only 4 has a transition on b, to 5.

```
Dtrans[A, b] = ε-closure( $\delta(\{1,2,4,7\}, b)$ )  
              = ε-closure( $\delta(0,b), \delta(1,b), \delta(2,b),$   
                           $\delta(4,b), \delta(7,b)$ )  
              = ε-closure(5) = {1,2,4,5,6,7}  
              let's call this state C.
```

```
Dtrans[A, b] = C
```

Now $Dtrans[B, a]$: it is similar to B itself, as only 2 and 7 have transitions on a, to 3 and 8 respectively.

```
Dtrans[B, a] = ε-closure(3, 8) = {1,2,3,4,6,7,8}  
Dtrans[B, a] = B
```

Subset Construction — States D and Back to B

```
Dtrans[B, b] =  $\epsilon$ -closure( $\delta(\{1,2,3,4,6,7,8\}, b)$ )  
              =  $\epsilon$ -closure( $\delta(1,b), \delta(2,b), \delta(3,b), \delta(4,b),$   
                           $\delta(6,b), \delta(7,b), \delta(8,b)$ )  
              =  $\epsilon$ -closure(5, 9) = {1,2,4,5,6,7,9}
```

```
let Dtrans[B, b] = D
```

```
Dtrans[C, a] =  $\epsilon$ -closure( $\delta(\{1,2,4,5,6,7\}, a)$ )  
              =  $\epsilon$ -closure(3, 8) = {1,2,3,4,6,7,8}  
              = B    (again similar to B)
```

```
Dtrans[C, b] =  $\epsilon$ -closure( $\delta(\{1,2,4,5,6,7\}, b)$ )  
              =  $\epsilon$ -closure(5, 4) = {1,2,4,5,6,7}  
              = C    (again similar to C)
```

Subset Construction — States D and E

```
Dtrans[D, a] =  $\epsilon$ -closure(3, 8)
              = {1,2,3,4,6,7,8}  similar to B
Dtrans[D, a] = B
```

```
Dtrans[D, b] =  $\epsilon$ -closure( $\delta(\{1,2,4,5,6,7,9\}, b)$ )
              =  $\epsilon$ -closure( $\delta(1,b), \delta(2,b), \delta(4,b), \delta(6,b),$ 
                           $\delta(7,b), \delta(9,b)$ )
              =  $\epsilon$ -closure(5, 10) = {1,2,4,5,6,7,10}
              let it be E
```

```
Dtrans[D, b] = E
```

```
Dtrans[E, a] =  $\epsilon$ -closure(3, 8) = {1,2,3,4,6,7,8}
              = B (similar to B)
```

Subset Construction — Summary

$$\begin{aligned} \text{Dtrans}[E, b] &= \varepsilon\text{-closure}(5) = \{1, 2, 4, 5, 6, 7\} \\ &= C \quad (\text{similar to } C) \end{aligned}$$

We got:

$$\begin{aligned} A &= \{0, 1, 2, 4, 7\} \\ B &= \{1, 2, 3, 4, 6, 7, 8\} \\ C &= \{1, 2, 4, 5, 6, 7\} \\ D &= \{1, 2, 4, 6, 7, 9\} \\ E &= \{1, 2, 4, 5, 6, 7, 10\} \end{aligned}$$

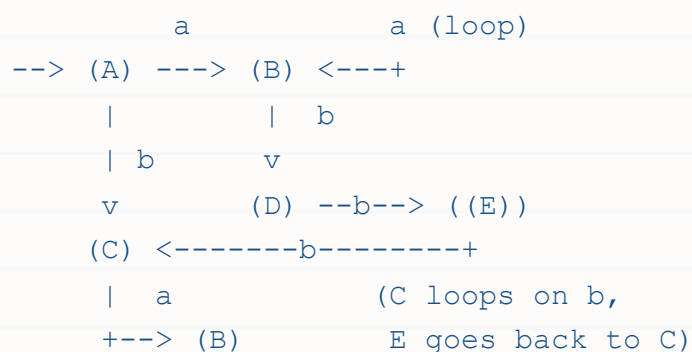
TRANSITION ON A	TRANSITION ON B
$\text{Dtrans}(A, a) = B$	$\text{Dtrans}(A, b) = C$
$\text{Dtrans}(B, a) = B$	$\text{Dtrans}(B, b) = D$
$\text{Dtrans}(C, a) = B$	$\text{Dtrans}(C, b) = C$
$\text{Dtrans}(D, a) = B$	$\text{Dtrans}(D, b) = E$
$\text{Dtrans}(E, a) = B$	$\text{Dtrans}(E, b) = C$

The Final DFA

A is the start state, and state E — which contains 10 — is the only accepting state. The final Dtrans is:

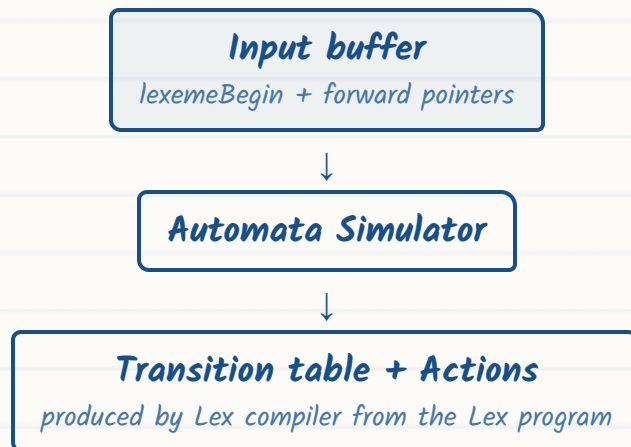
NFA STATES	DFA STATE	A	B
{0,1,2,4,7}	A	B	C
{1,2,3,4,6,7,8}	B	B	D
{1,2,4,5,6,7}	C	B	C
{1,2,4,6,7,9}	D	B	E
{1,2,4,5,6,7,10}	E	B	C

The DFA for the above NFA using subset construction:



Design of a Lexical Analyzer Generator

1. Structure of the Generated Analyzer



- This structure describes how a finite automata simulator works for the lexical analyzer.
- The Lex program simulates an automaton.
- The Lex program consists of all the regular-expression definitions and translation rules.
- Once a Lex program is compiled, the regular expressions are converted into finite automata.
- The translation rules are turned into a transition table and actions.

Automata Simulator

The transition table and actions are used by the finite automata simulator to recognize the tokens. The simulator checks whether any input strings stored in the input buffer match any pattern; if one matches, it produces the corresponding token as output.

The automata simulator uses the following components:

- A transition table for the automaton.
- Functions that are passed directly through Lex to the output.
- The actions from the input, which appear as fragments of code to be invoked at the appropriate time by the automata simulator.

1. To Construct an Automaton

- Convert the regular-expression patterns in the Lex program to NFAs.
- We need a single automaton that will recognize lexemes matching any of the patterns in the program.

Combining the NFAs

We combine all the NFAs into one by introducing a new start state with ϵ -transitions. Example:

```
a      { action A1, for pattern P1 }
abb    { action A2, for pattern P2 }
a*b+   { action A3, for pattern P3 }
```

NFAs for *a*, *abb*, and *a*b+*:

```
start --> (1) -a-> ((2))

start --> (3) -a-> (4) -b-> (5) -b-> ((6))

start --> (7) -b-> ((8))
           a loop      b loop
```

Combined NFA:

```

      ε      (1) -a-> ((2))
      /
--> (0) -ε- (3) -a-> (4) -b-> (5) -b-> ((6))
      \
      ε      (7) -b-> ((8))
```

Pattern Matching Based on the NFA

Input string *abba* — pattern matching based on the NFA:

```
{0,1,3,7} -a-> {2,4,7} -a-> {7} -b-> {8} -a-> NONE
```

This is the sequence of sets of states entered when processing input abba.

The DFA for the above NFA:

```
start --> (0137) --a--> ((247))
      |           |
      b           b
      v           v
((8)) <--b-- ((68)) <--b-- (0)
b loop      a*b+ accepting
```

*Transition graph for the DFA handling the patterns *a*, *abb*, and *a*b+*.*

Optimization of DFA-Based Pattern Matching

Convert the regular expression into a DFA without creating an NFA:

- *Introduce the augmented regular expression.*
- *Construct a syntax tree for the RE.*
- *Number the leaf nodes.*
- *Traverse the syntax tree to compute the function nullable.*
- *Compute firstpos and lastpos for each node.*
- *Compute followpos of each node.*
- *Convert to a DFA.*